



Universidad ORT Uruguay
Facultad de Ingeniería

OBLIGATORIO
ESTRUCTURA DE DATOS Y ALGORITMOS 2

Mateo Ferreira (282200)
Matías Piñeyro (271362)

2025

Ejercicios

Ejercicio 6

Para resolver este ejercicio, lo que pensamos inicialmente era si debíamos utilizar algo de grafos pero no era así, entendimos que debíamos usar “structs” con sus respectivas variables para atacar el problema, los subproblemas que se nos ocurrieron fueron las siluetas de los edificios del lado izquierdo y del lado derecho, ya que debíamos aplicar Divide and Conquer, y planteamos un caso base como en los ejercicios vistos en clase. Un problema que se nos presentó es que primero utilizamos una variable `&cantRes` para manejar la cantidad de pares que íbamos a devolver al final, como en realidad el uso de “&” no es recomendable lo sustituimos por el uso de una struct `Resultado` que tenía un puntero `Par` y un entero cantidad resolviendo así nuestro problema.

Lo complicado también fue hacer los casos del merge en la función auxiliar “silueta” ya que teníamos que considerar si la altura izquierda era mayor a la derecha, o si la derecha era mayor o si eran iguales. Y al final corroborar si todavía quedaban algunos que se nos hubieran pasado.

Su orden temporal se debe a que cuando aplico divide and conquer y hago 2 llamadas recursivas, una para la parte izquierda y otra para la parte derecha, ahí subdivido el problema en $N/2$ pero cuando las uno me queda $2T(N/2)$. Luego todos los whiles que hago representan $O(N)$, cuando sumamos nos queda $O(N * \log(N))$.

Para este ejercicio hicimos muy poco uso de ChatGPT(<https://openai.com/es-ES/>), solo lo utilizamos para corregir errores como “std::bad_alloc” o “std::bad_array_new_length”.

Ejercicio 7

Para resolver este ejercicio lo fuimos armando mediante prompts de ChatGPT pero sin pedirle el código explícitamente, sino que lo fuimos armando con él para ver si íbamos en el camino correcto(pedíamos pistas). El mayor problema que teníamos era cómo aplicar la lógica de que los números tuvieran una frecuencia impar o par. Al entender eso, supimos que si se presentaba un dígito que tuviera frecuencia impar más de 1 vez ya devolveremos que “No” es palíndromo. Otra cosa que investigamos es como pasar de enteros a caracteres ya que lo que queríamos devolver era un string y se hacía mediante un casteo (char) a la suma de ‘0’ a algún entero. Con eso pudimos resolver cuando poner el dígito de frecuencia impar en el medio y como agregarle la mitad izquierda y su reverso.

Su orden temporal se debe a que hacemos todo el ejercicio del “tirón” por eso es greedy, entonces queda con $O(N)$, la única parte que puede llegar a confundir que no cumple con $O(N)$ es cuando armo el palíndromo y tengo 1 for anidado, pero en realidad lo que sucede es que en el primer for siempre voy a iterar 10 dígitos del mayor al menor lo que tiene $O(1)$ y el segundo for es el que se ejecuta a lo sumo N veces. Por ende tiene $O(N)$.

Ejercicio 8

Para resolver este ejercicio lo que hicimos fue primero pensarlo recursivamente como hicimos en clase y luego lo pasamos a Memoization. La parte recursiva fue la que más nos costó ya que hubieron ciertas cosas que las fuimos aclarando a medida que lo íbamos resolviendo y no habían quedado claras de entrada. Una de las principales fue llegar a la lógica esa de pensarlo con las dos opciones (A y B) que hay posibles. Dado que por letra nos dice que el asterisco seguido de un carácter representa a 0 o más del carácter anterior, nosotros evaluamos el caso en que se usa cero veces, osea que lo saltea al carácter junto al asterisco y compara el siguiente con el carácter del texto. Y el otro caso que sería el que comparo si coincide ese carácter con el del texto, y como tiene asterisco, sigo chequeando si hay más repeticiones del mismo que se contemplen bajo ese asterisco.

En conclusión, si nos da true la opción de usar cero veces significa que ignorando completamente el carácter con el '*', el patrón estaría cumpliendo con el texto de todos modos. En el otro caso evalúa si el primero coincide junto a la recursiva avanzando uno en el texto, dejando para poder seguir comparando con ese patrón. Sin ser esta parte el resto lo pudimos razonar bastante bien. Y para pasarlo a memo añadimos la matriz inicializada en -1 y comprobamos cuando entra en coincideRec(i,j) si ya existe un valor en la matriz con fila i y columna j, y lo devolvemos así nos evitamos recorrer de más y hacer consultas ya hechas.

La complejidad nos queda en $O(N \cdot M)$ tanto temporal como espacial tal como pide la letra. El espacial es $O(N \cdot M)$ porque guardamos una tabla memo de tamaño $(n+1) \times (m+1)$ para almacenar el resultado de cada subproblema. El temporal es $O(N \cdot M)$ porque con memoización cada uno de los $N \cdot M$ posibles estados tanto en el texto como en el patrón, se calcula como máximo una vez y cualquier acceso que sea posterior recupera el resultado en $O(1)$.

Ejercicio 9

Para este ejercicio, luego de consultas con los profesores y visto los ejercicios en clase, dedujimos que podíamos utilizar el problema de la mochila para resolverlo. Lo que hicimos fue adaptar el problema de la mochila que vimos en clase de 2 dimensiones pero para 3 dimensiones como pide la letra de este problema.

Su orden temporal se debe a que uso una programación dinámica tridimensional. Definí una tabla de dimensiones $N + 1, S + 1$ y $L + 1$ por lo que la inicialización de la tabla toma tiempo $O(N \cdot S \cdot L)$. Luego, para llenar la tabla de nuevo se usa un triple for, por ende queda que su orden espacial y temporal es $O(N \cdot S \cdot L)$.

Hicimos uso de ChatGPT para evacuar alguna duda que nos dio cuando le agregamos la 3era dimensión que al principio no nos estaba quedando bien y nos tiraba errores por ejemplo "Segmentation fault"

Ejercicio 10

Para este problema, al tener conocimientos ya sobre este juego la parte del funcionamiento ya se nos hizo más sencilla de entender. Realizamos alguna consulta en GeeksForGeeks que tenían un ejemplo similar pero no copiamos nada de ahí. A lo que en backtracking tenemos un esquema de solución más concreto y no tan abstracto se hizo más llevadero aunque fue largo. Surgieron algunas consultas que descartamos por Teams y en clase sobre los bloques y si siempre eran fijo 9 bloques. Tuvimos pocos errores, la mayoría de tipos o de implementación en C++ que los resolvimos bien y algunos los consultamos con ChatGPT.

Resultados

Ejercicio	Resultado
6	Completo
7	Completo
8	Completo
9	Completo
10	Completo

- Completo: Corren todas las pruebas.
- Parcial: Corren parcialmente las pruebas.
- En Proceso: Implementado pero no pasa ninguna prueba.
- No Implementado: Ejercicio no implementado o no compila.